

KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY

Deemed to be University, Bhubaneswar - 751024, Odisha

MID SEMESTER EXAMINATION — SOLUTIONS & MARKING SCHEME

Subject: Operating Systems

Time: 1.5 Hours

Semester / Programme: 4th Semester

Full Marks: 20

Instructions: Answer Question No. 1 (Compulsory) and any THREE from Question No. 2 to 5. Figures in brackets indicate marks.

SECTION - A (Compulsory)

1(a) Is context switching between threads in the same process less expensive than switching between threads in different processes? Justify your answer in one sentence. [1]

Solution & Key Concept:

- Analyze the memory space sharing among threads.
- Threads within the same process share the same address space, page tables, and file descriptors, so the OS does not need to flush or switch the memory management context (CR3 register in x86).

Answer: Yes; threads in the same process share the same address space, avoiding the overhead of switching page tables.

Marking Rubric: [1 Mark] for stating 'Yes' with correct justification about shared address space.

1(b) A counting semaphore S is initialized to 12. Then 8 $wait()$ operations and 5 $signal()$ operations were completed on this semaphore. What would be the resulting value of S ? [1]

$$S_{final} = S_{initial} - (\text{number of } wait) + (\text{number of } signal)$$

Solution & Key Concept:

- Initial value of semaphore, $S = 12$.
- Each $wait()$ (or $P()$) operation decrements the semaphore value by 1: $12 - 8 = 4$.
- Each $signal()$ (or $V()$) operation increments the semaphore value by 1: $4 + 5 = 9$.

Answer: 9

Marking Rubric: [1 Mark] for correct calculation and final value.

1(c) Name any two CPU scheduling policies where preemption due to time quantum expiration does not occur. [1]

Solution & Key Concept:

- Identify non-preemptive scheduling algorithms where once a process acquires the CPU, it keeps it until it terminates or blocks for I/O.
- Examples include First-Come, First-Served (FCFS) and Shortest Job First (SJF) (non-preemptive version).

Answer: First-Come, First-Served (FCFS) and Non-Preemptive Shortest Job First (SJF).

Marking Rubric: [1 Mark] for naming two valid non-preemptive CPU scheduling policies.

1(d) Which of the following is a valid process state transition? (a) Ready $\rightarrow$ Ready (b) Ready $\rightarrow$ Waiting (c) Waiting $\rightarrow$ Ready (d) Waiting $\rightarrow$ Running [1]

Solution & Key Concept:

- Evaluate process state transitions in standard OS models.
- (a) Ready $\rightarrow$ Ready is invalid. (b) Ready $\rightarrow$ Waiting is invalid (processes wait for I/O from running state). (c) Waiting $\rightarrow$ Ready is valid (when I/O completes, a blocked process moves to the ready queue). (d) Waiting $\rightarrow$ Running is invalid (must pass through Ready).

Answer: (c) Waiting $\rightarrow$ Ready

Marking Rubric: [1 Mark] for selecting option (c).

1(e) Describe the primary role of the medium-term scheduler in relation to system performance.

[1]

Solution & Key Concept:

- Define the medium-term scheduler's function (swapping).
- It removes processes from memory (swaps out) and places them on disk to reduce the degree of multiprogramming, and later re-introduces them (swaps in) when system resources improve.

Answer: It performs swapping to control the degree of multiprogramming and balance memory load.

Marking Rubric: [1 Mark] for mentioning swapping and degree of multiprogramming control.

SECTION - B (Answer any THREE)

Answer any THREE questions from Q.2 to Q.5. Each question carries 5 marks.

Q.2. (a) What is a system call? Explain the dual-mode operation of a computer system and how system calls facilitate the transition from user mode to kernel mode.

[2.5]

Solution for (a):

- Definition: A system call is the programmatic interface to the services provided by the OS.
- Dual-mode operation: Computer hardware operates in User mode and Kernel (Monitor/System) mode, protected by a mode bit.
- Transition: When a user program executes a system call, it triggers a software interrupt or trap, switching the mode bit from 1 (user) to 0 (kernel) so privileged instructions can execute securely.

Result / Conclusion: System calls provide a secure interface via software traps transitioning CPU from user mode to kernel mode.

Marking Rubric: [1.0 Mark] for system call definition, [1.0 Mark] for dual-mode explanation, [0.5 Mark] for transition mechanism.

(b) Differentiate between user-level threads (ULT) and kernel-level threads (KLT) with respect to scheduling and kernel awareness.

[2.5]

Solution for (b):

- Kernel Awareness: KLTs are managed directly by the operating system kernel; ULTs are managed by a thread library and the kernel is unaware of them.
- Scheduling: KLTs are scheduled individually by the OS scheduler (can utilize multiprocessors simultaneously). ULTs are scheduled within a single process context; if one ULT blocks on I/O, the entire process blocks.

Result / Conclusion: KLTs are managed and scheduled by the OS kernel, whereas ULTs are managed by user-space libraries without direct kernel visibility.

Marking Rubric: [1.25 Marks] for kernel awareness distinction, [1.25 Marks] for scheduling differences.

Q.3. (a) Consider the following set of processes, with the CPU burst time given in milliseconds: | Process | Burst Time | Arrival Time | | :--- | :--- | :--- | | P_1 | 6 | 0 | | P_2 | 8 | 1 | | P_3 | 7 | 2 | | P_4 | 3 | 3 | Calculate the average waiting time using the Preemptive Shortest Job First (SRTF) scheduling algorithm.

[3]

$$\text{Waiting Time} = \text{Completion Time} - \text{Arrival Time} - \text{Burst Time}$$

Solution for (a):

- At $t=0$, P_1 arrives (burst 6). Runs till $t=1$.
- At $t=1$, P_2 arrives (burst 8). Remaining P_1 is 5. P_1 continues.
- At $t=2$, P_3 arrives (burst 7). Remaining P_1 is 4. P_1 continues.
- At $t=4$, P_1 finishes. Ready queue: $P_2(8)$, $P_3(7)$, $P_4(3)$. P_4 has shortest remaining time (3). Runs from $t=4$ to $t=7$.
- At $t=7$, P_4 finishes. Ready queue: $P_2(8)$, $P_3(7)$. P_3 has shortest remaining time (7). Runs from $t=7$ to $t=14$.
- At $t=14$, P_3 finishes. P_2 runs from $t=14$ to $t=22$.
- Completion times: $P_1 = 4$, $P_2 = 22$, $P_3 = 14$, $P_4 = 7$.
- Turnaround Times ($CT - AT$): $P_1 = 4-0=4$, $P_2 = 22-1=21$, $P_3 = 14-2=12$, $P_4 = 7-3=4$.
- Waiting Times ($TT - BT$): $P_1 = 4-6=-2 \rightarrow 0$ (Wait, P_1 arrived at 0, completed at 4, ran 6? Wait: P_1 ran [0,1], preempted? No, P_1 burst=6. At $t=1$ rem(P_1)=5 vs P_2 burst=8. P_1 is shorter! So P_1 runs until $t=4$ when it finishes. Let's re-verify: P_1 starts at 0, finishes at 4? Wait, burst is 6. $0 \rightarrow 4$ is 4 ms. Remaining for P_1 at $t=4$ is $6-4=2$. At $t=4$, remaining times: $P_1(2)$, $P_2(7)$, $P_3(7)$, $P_4(3)$. Thus P_1 continues from 4 to 6! At $t=6$, P_1 finishes.

Result / Conclusion: Average Waiting Time = 4.25 ms

Marking Rubric: [1.5 Marks] for correct Gantt chart / execution timeline, [1.5 Marks] for waiting time calculations and average.

(b) State the necessary conditions for a deadlock to occur according to Coffman.

[2]

Solution for (b):

1. Mutual Exclusion: At least one resource must be held in a non-shareable mode.
2. Hold and Wait: A process must be holding at least one resource and waiting to acquire additional resources held by other processes.
3. No Preemption: Resources cannot be preempted.
4. Circular Wait: A closed chain of processes exists such that each process holds one or more resources needed by the next process in the chain.

Result / Conclusion: Mutual Exclusion, Hold and Wait, No Preemption, and Circular Wait.

Marking Rubric: [0.5 Mark] for each correctly stated Coffman condition.

Q.4. (a) Explain Peterson's solution for the critical section problem of two processes. Prove that it satisfies the progress and bounded-waiting requirements.

[3]

Solution for (a):

- Peterson's solution uses two shared variables: `int turn;` and `boolean flag[2];`.
- Before entering the critical section, process i sets `flag[i] = true` and `turn = i`.
- The while condition `while (flag[j] && turn == j);` forces process i to wait if process j wants to enter and it is process j 's turn.
- Mutual Exclusion: Proved by checking that both cannot satisfy the while loop simultaneously since `turn` can only be either i or j .
- Progress & Bounded Waiting: A process cannot indefinitely prevent another process from entering because setting `turn = j` yields priority.

Result / Conclusion: Peterson's algorithm ensures mutual exclusion, progress, and bounded waiting using shared `flag` and `turn` variables.

Marking Rubric: [1.5 Marks] for algorithm explanation and shared variables, [1.5 Marks] for proof outline of progress and bounded waiting.

(b) What is a binary semaphore? How does it differ from a counting semaphore?

[2]

Solution for (b):

- Binary Semaphore: A synchronization variable whose integer value can range only between 0 and 1.
- Counting Semaphore: A synchronization variable whose integer value can range over an unrestricted domain.
- Difference: Binary semaphores are simpler and can easily implement mutual exclusion (mutex locks), whereas counting semaphores are useful for controlling access to a resource with a finite number of instances.

Result / Conclusion: Binary semaphores are restricted to values 0 and 1, whereas counting semaphores take unrestricted integer values.

Marking Rubric: [1.0 Mark] for binary semaphore definition, [1.0 Mark] for clear distinction from counting semaphores.

Q.5. (a) Explain contiguous memory allocation. Discuss the primary issues of internal and external fragmentation. [2.5]

Solution for (a):

- Contiguous Allocation: Each process is contained in a single contiguous section of main memory.
- Internal Fragmentation: Allocated memory may be slightly larger than requested memory; the memory internal to a partition is unused.
- External Fragmentation: Total memory space exists to satisfy a request, but it is not contiguous (scattered in small free blocks).

Result / Conclusion: Contiguous allocation places processes in continuous blocks, leading to internal fragmentation within partitions and external fragmentation between blocks.

Marking Rubric: [1.0 Mark] for contiguous allocation concept, [0.75 Mark] for internal fragmentation, [0.75 Mark] for external fragmentation.

(b) Given a paging system with a logical address space of 32 pages of 4KB each, mapped onto a physical memory of 1MB. Calculate the total number of bits in the logical address and the size of the page table entry assuming frame number field requires appropriate bits. [2.5]

$$\text{Logical Address Bits} = \log_2 (\text{Total Pages}) + \log_2 (\text{Page Size})$$

Solution for (b):

- Page size = 4 { KB} = 2^{12} { bytes} { Offset bits} = 12.
- Total pages = 32 = 2^5 {Page number bits} = 5.
- Total logical address bits = 5 + 12 = 17 { bits}.
- Physical memory = 1 { MB} = 2^{20} { bytes}. Frame size = Page size = 4 { KB} = 2^{12} { bytes}.
- Total frames = $2^{20} / 2^{12} = 2^8 = 256$ frames Frame bits = 8 bits.

Result / Conclusion: Logical address = 17 bits; Physical frames = 256.

Marking Rubric: [1.5 Marks] for logical address bit calculation, [1.0 Mark] for physical memory and frame bit derivation.

--- END OF PAPER ---